Mossbound

---

1. Концепція гри

1.1. Назва

Mossbound

1.2. Загальна ідея

Mossbound - це 2D-платформер про дослідження та подолання перешкод у загадковому магічному болоті. Гравець керує мандрівником, який прокидається біля покинутої стежки та намагається дістатися стародавнього джерела світла в глибині болота.

Світ складається з мохових платформ, гігантських синіх грибів, старого коріння, ліан, туманних проваль. Навколишнє середовище створює відчуття таємничості й небезпеки, основний виклик гри полягає в опануванні руху.

Персонаж повинен навчитися долати широкі проміжки, змінювати траєкторію в повітрі, відштовхуватися від стін і поєднувати різні рухові можливості для проходження складніших ділянок.

1.3. Основна ігрова мета

Дістатися до кінця тестового рівня, використовуючи всі доступні механіки руху.

1.4. Цільовий ігровий досвід

Гравець має відчувати, що персонаж:

- швидко реагує на натискання кнопок;
- плавно прискорюється та сповільнюється;
- має зрозумілу й передбачувану траєкторію стрибка;
- може коригувати напрямок руху в повітрі;
- природно поєднує стрибок, подвійний стрибок, Dash і Wall Jump;
- дає змогу виправити невелику помилку без необхідності щоразу починати проходження заново.

Основний принцип дизайну: рух має бути плавним, контрольованим.

---

2. Візуал

2.1. Художній напрям

Гра використовує: https://theflavare.itch.io/forest-nature-fantasy-tileset

Основні елементи середовища:

- болотний ліс;
- великі гриби;
- платформи, вкриті мохом;
- стародавні корені та звисаючі ліани;
- кілька шарів паралаксу для створення глибини.

2.2. Персонаж

Головний герой - невеликий мандрівник, який досліджує болото.

Його спрайт повинен залишатися читабельним на тлі оточення. Анімації руху мають допомагати гравцеві розуміти стан персонажа: коли він стоїть, розганяється, перебуває в повітрі, виконує Dash або відштовхується від стіни.

---

3. Дизайн руху персонажа

3.1. Основні принципи

Система руху повинна відповідати стилю Fluid Platforming - плавного платформінгу, в якому гравець зберігає контроль над персонажем під час руху на землі та в повітрі.

Основні принципи:

- **Responsiveness** - швидка реакція. Персонаж повинен оперативно реагувати на введення гравця.
- **Fluidity** - плавність. Механіки мають поєднуватися без різких і непередбачуваних переходів.
- **Air Control** - контроль у повітрі. Гравець повинен мати можливість коригувати траєкторію після стрибка.
- **Forgiveness** - поблажливість до помилок. Невеликі неточності можна виправляти за допомогою повітряного контролю та додаткових механік.
- **Consistency** - передбачуваність. Однакові дії за однакових умов повинні давати приблизно однаковий результат.

З Forgiveness не вийшло, місцями на перших рівнях дуже складно

3.2. Основний набір механік

№	Механіка	Призначення
1	Move	Горизонтальне переміщення персонажа
2	Jump	Основний стрибок
3	Double Jump	Додатковий стрибок у повітрі
4	Dash	Короткий швидкий ривок
5	Wall Jump	Стрибок із відштовхуванням від стіни

Ці п’ять механік є основою ігрового процесу. Кожна повинна мати чітке призначення та використовуватися в тестовому рівні.

---

4. Детальна специфікація механік

4.1. Move - горизонтальний рух

Мета: забезпечити плавне, точне та чутливе горизонтальне переміщення.

Правила роботи

- Персонаж рухається вліво та вправо.
- Після натискання напрямку персонаж прискорюється до максимальної швидкості.
- Після відпускання кнопки персонаж поступово сповільнюється.
- Під час руху в повітрі гравець може змінювати горизонтальний напрямок.
- Зміна напрямку не повинна миттєво телепортувати персонажа або створювати неприродний рух.
  Відсутність розділових знаків трошки ускладнює розуміння тексту

- Після приземлення персонаж повинен зберігати можливість одразу рухатися або стрибати.
- Поворот спрайта має відповідати напрямку руху або останньому заданому напрямку.

Керування

Дія	Керування
Рух ліворуч	A або ←
Рух праворуч	D або →

Обов’язково додайте підтримку гемпаду

Початкові параметри

Параметр	Значення	Призначення
`maxMoveSpeed`	7	Максимальна горизонтальна швидкість
`groundAcceleration`	45	Прискорення на землі
`groundDeceleration`	55	Гальмування на землі
`airAcceleration`	30	Прискорення в повітрі

Великий плюс, що ти працюєш з едітором, але ці значення будуть залежати як від реалізації стрибку так і від розмірів асетів а ігровому полі. Зачасту на проєктах в тебе до цього доступу не буде. Тому зазвичай:
1.Розробка механіки по рефу/опису.
2.Тюнинг значень на прототипі

Одиниці швидкості - Unity units/s, прискорення - units/s².

AC:

- Персонаж рухається в обох напрямках.

- Прискорення та гальмування помітні, але не створюють відчуття ковзання.
- Гравець може скоригувати приземлення в повітрі.
- Рух не залежить від частоти кадрів.
- Після приземлення немає небажаної затримки перед наступною дією.

4.2. Jump - основний стрибок

Мета: забезпечити передбачуваний стрибок, висоту якого гравець може частково контролювати.

Правила роботи

Це по факту одне правило: висота стрибка залежить від довжини натисненя

- Стрибок можна активувати, коли персонаж переб
- Стрибок починається відразу після прийняття команди.
- Якщо гравець утримує кнопку стрибка, персонаж може стрибнути вище.
- Якщо гравець відпускає кнопку раніше, висота стрибка зменшується.
- Траєкторія має виразну фазу підйому, вершину та падіння.
- Гравець може керувати горизонтальним рухом протягом усього стрибка.
- Після приземлення стрибок знову стає доступним.

Для реалізації змінної висоти стрибка програмістка може зменшувати вертикальну швидкість після раннього відпускання кнопки. Конкретний алгоритм потрібно узгодити.

Керування

Дія	Керування
Стрибок	Space
Короткий стрибок	Коротке натискання Space
Вищий стрибок	Утримання Space

Початкові цілі

Параметр	Ціль

`targetJumpHeight`	Приблизно 2,5 units
`targetTimeToApex`	Приблизно 0,35 s
`jumpBufferTime`	0,10 s
`coyoteTime`	0,10 s

`jumpBufferTime` - час, протягом якого гра запам’ятовує натискання стрибка перед приземленням.

`coyoteTime` - короткий проміжок після сходження з платформи, протягом якого стрибок ще можна виконати.

Ці два параметри є рекомендованими налаштуваннями для першого прототипу.

Похідні параметри:

Параметр	Значення	Призначення
`gravity`	$2 \cdot h \ / \ t^2 \approx$ 41 units/s²	Виводиться з `targetJumpHeight` і `targetTimeToApex`
`jumpVelocity`	$\texttt{gravity} \cdot t \approx 14,3$ units/s	[illegible]
`fallGravityMultiplier`	1,5 (≈ 61 units/s² на падінні)	[illegible] швидше за підйом
`maxFallSpeed`	16 units/s	Обмеження швидкості падіння

Виглядає більше як документація чим технічне завдання. По назвам можна здогадатися, що і куди, але якщо маєш в голові конкретний алгоритм - варто його описати.

`jumpCutGravityMultiplier`	3	Мінімальна висота короткого стрибка ≈ 1,0 unit

Орієнтири: повний час у повітрі ≈ 0,64 с; дальність одного стрибка на `maxMoveSpeed` ≈ **4,4 units**.

AC:

- Коротке та тривале натискання дають помітно різну висоту (≈ 1,0 і ≈ 2,5 units).
- Персонаж не зависає у верхній точці.
- Гравець може коригувати горизонтальну траєкторію.
- Після приземлення можна одразу виконати новий стрибок.

4.3. Double Jump - подвійний стрибок

Мета: дозволити гравцеві виправляти траєкторію в повітрі та досягати вищих платформ.

Правила роботи

- Після основного стрибка персонаж отримує право на один додатковий стрибок.
- Під час активації вертикальна швидкість змінюється відповідно до налаштувань механіки.
- Другий стрибок повинен бути помітним і давати можливість змінити траєкторію.
- Після використання Double Jump повторна активація неможлива до приземлення на валідну поверхню.

Керування

Дія	Керування
Основний стрибок	Space

Подвійний стрибок	Повторне натискання Space у повітрі

Початкові параметри

Параметр	Значення	Призначення
`doubleJumpVerticalSpeed`	11,5 units/s	Вертикальна швидкість після активації
`maxAirJumps`	1 додатковий стрибок	Кількість повітряних стрибків

AC:

- Double Jump можна виконати в будь-який момент повітряної фази.
- Double Jump можна виконати після Wall Jump.
- Подвійний стрибок допомагає досягати платформ, недоступних для звичайного стрибка (висота до ≈ 4,1 unit).
- Механіка не створює нескінченного стрибка через помилкове визначення землі.

4.4. Dash - швидкий ривок

Мета: надати персонажу можливість швидко долати проміжки та змінювати позицію під час руху.

Dash повинен відчуватися як короткий імпульс, а не як звичайний рух зі збільшеною швидкістю.

Правила роботи

- Dash можна активувати на землі та в повітрі.
- Напрямок визначається введенням гравця.
- Якщо гравець не задає напрямок, Dash виконується в напрямку, куди повернутий персонаж.
- Дозволено горизонтальний, вертикальний та діагональний Dash.
- Під час Dash персонаж отримує контрольований імпульс швидкості.
- Після завершення Dash персонаж повертається до звичайного руху.

Це як?

- У першій версії дозволено один Dash на одну повітряну послідовність.
- Заряд Dash відновлюється після приземлення.
- Dash можна комбінувати зі звичайним стрибком, Double Jump і Wall Jump.

єдині правила:

Все ще про вертикаль й діагональ не зрозумів, в грі наче не було

- Вектор напрямку **нормалізується**, тому швидкість однакова в усіх напрямках, а діагональний Dash не швидший за горизонтальний (так само дистанція ≈ 3,5 unit).
- Наземний Dash: вертикаль «вниз» ігнорується (Dash виконується в напрямку погляду); вертикаль «вгору» або діагональ угору дозволені й відривають персонажа від землі.
- Під час Dash гравітація й керування в повітрі вимкнені (`dashGravityScale = 0`); у кінці Dash вертикальна швидкість обнуляється, щоб Dash вгору не давав зайвої висоти.
- Dash не можна почати під час іншого Dash.

Керування

Дія	Керування
Dash	Left Shift (додатково K)
Напрямок Dash	A/D, ←/→ та/або W/S, ↑/↓

Для прототипу: за відсутності вертикального введення Dash виконується горизонтально, а за наявності вертикального введення враховується заданий напрямок. Якщо напрямок не задано, використовується напрямок погляду персонажа.

Початкові параметри

Параметр	Значення	Призначення
`dashDuration`	0,16 s	Тривалість Dash

`dashSpeed`	22 units/s	Швидкість під час Dash (≈ 3,5 unit за 0,16 s)
`maxAirDashes`	1	Повітряні Dash до приземлення
`dashCooldown`	0,30 s (лише наземний Dash)	Пауза між наземними Dash
`dashGravityScale`	0	Гравітація під час Dash
`dashExitSpeed`	8 units/s	Горизонтальна швидкість у момент завершення
`dashExitDecayTime`	0,20 s	За скільки секунд швидкість спадає до `maxMoveSpeed`
`dashBufferTime`	0,10 s	Запам’ятовування натискання Dash

Повернення до звичайного руху

Після завершення Dash потрібно уникнути двох крайнощів:

- персонаж миттєво зупиняється, хоча рухався на великій швидкості;
- персонаж зберігає надмірну швидкість і постійно проскакує потрібні платформи.

Для прототипу у момент завершення горизонтальна швидкість дорівнює `dashExitSpeed` (8) у напрямку Dash і лінійно спадає до `maxMoveSpeed` за `dashExitDecayTime`; керування `airAcceleration` повертається одразу. Після суто вертикального Dash горизонтальна швидкість залишається такою, як була до Dash.

Орієнтири дальності: Jump + Dash ≈ **7,9 units**; Jump + Double Jump + Dash ≈ **10,5 units**.

AC:

- Ривок має чітку тривалість і передбачувану дистанцію (≈ 3,5 unit у будь-якому з напрямків).
- Діагональний Dash не швидший за горизонтальний.
- Після Dash гравець знову контролює персонажа.
- Dash не дозволяє проходити крізь звичайні тверді платформи та стіни.
- Повітряний Dash доступний лише один раз до приземлення; наземний Dash має cooldown 0,30 s.

4.5. Wall Jump - стрибок від стіни

Мета: забезпечити контрольоване вертикальне переміщення та проходження ділянок із високими стінами.

Правила роботи

- Wall Jump доступний, коли персонаж перебуває в повітрі та торкається валідної стіни.
- Після натискання Jump персонаж отримує імпульс угору та від стіни.
- Напрямок початкового відштовхування визначається стороною стіни.
- Після початкового імпульсу гравець може коригувати горизонтальний рух.
- Wall Jump можна поєднувати з Double Jump.
- Після відштовхування від стіни повторний Wall Jump не повинен спрацьовувати без нового валідного контакту.
- Після приземлення персонаж повертається до звичайного стану руху.

Керування

Дія	Керування
Стрибок від стіни	Space під час контакту зі стіною
Відхилення після стрибка	A/D або ←/→

Початкові параметри

Параметр	Значення	Призначення
`wallJumpHorizontalSpeed`	8 units/s	Горизонтальна швидкість від стіни
`wallJumpVerticalSpeed`	13 units/s	Вертикальна швидкість
`wallCheckDistance`	0,06 unit (≈ 1 px)	Відстань від краю колайдера до стіни
`wallJumpControlLockTime`	0,08 s (діапазон тесту 0–0,12 s)	Блокування вводу в бік стіни

Коротке обмеження горизонтального керування після відштовхування - лише варіант для тестування. Якщо воно робить рух менш плавним, його можна зменшити або повністю прибрати.

AC:

- Wall Jump спрацьовує тільки за валідного контакту зі стіною.
- Персонаж відштовхується в правильному напрямку.
- Повторний Wall Jump не спрацьовує без нового контакту (протилежна стіна або земля).
- Гравець може керувати персонажем після відштовхування.
- Механіка дозволяє підніматися вертикальними ділянками (≈ 2,1 unit за відскок).
- Виявлення стіни працює стабільно з обох боків персонажа.

4.6. Wall Contact - контакт зі стіною

Знову ж таки, Unity - великий плюс, але ти маєш описувати все на high-level не вдаючись в деталі. Бо не факт, що твоя компанія Unity для роботи буде використовувати. Це залишай розробникам.

Мета: визначити стан, у якому дозволе
не можна реалізувати однозначно.

Валідна стіна: колайдер із шару `Wall` з майже вертикальною поверхнею (нормаль по горизонталі); перевірка - коротким `BoxCast` збоку від колайдера на `wallCheckDistance`.

Умови входу: персонаж у повітрі, торкається валідної стіни, тримає ввід у бік стіни (`wallClingRequiresInput = true`), вертикальна швидкість ≤ 0.

Поведінка в стані: персонаж повільно сповзає зі швидкістю `wallSlideSpeed`; Double Jump і Dash не витрачаються; Jump → Wall Jump; Dash у цьому стані за замовчуванням недоступний (`allowDashFromWall = false`).

Умови виходу: приземлення → Grounded; ввід у бік стіни припинено або стіна закінчилась → Falling (після `wallCoyoteTime`, щоб Wall Jump ще можна було виконати); Wall Jump → Wall Jump.

Параметр	Значення	Призначення
`wallSlideSpeed`	2 units/s	Швидкість сповзання
`wallCoyoteTime`	0,10 s	Вікно Wall Jump після втрати контакту
`wallClingRequiresInput`	true	Чи потрібен ввід у бік стіни
`allowSameWallRepeat`	false	Заборона Wall Jump двічі від тієї самої сторони
`allowDashFromWall`	false	Dash зі стану Wall Contact

AC:

- Контакт визначається з обох боків персонажа й не спрацьовує від підлоги та стелі.
- Сповзання має постійну швидкість `wallSlideSpeed`.
- Wall Jump доступний протягом `wallCoyoteTime` після втрати контакту.
- Правило чергування стін (4.5) діє у стані Wall Contact.

---

5. Таблиця переходів між механіками

Рух повинен працювати як одна система, а не як п’ять незалежних компонентів.

Поточний стан	Дія	Очікуваний результат
Grounded	Jump	Починається основний стрибок
Jumping	Jump	Виконується Double Jump
Jumping	Dash	Починається Dash
Double Jump	Dash	Починається Dash
Jumping	Контакт зі стіною	Дозволяється Wall Jump
Double Jump	Контакт зі стіною	Дозволяється Wall Jump
Wall Contact	Jump	Виконується Wall Jump
Wall Jump	Jump	Виконується Double Jump
Wall Jump	Dash	Виконується Dash
Dash	Завершення тривалості	Повернення до звичайного контролю в повітрі

Будь-який повітряний стан	Приземлення	Grounded
Grounded	Jump після приземлення	Новий основний стрибок

Grounded	Dash	Наземний Dash (cooldown 0,30 s)
Grounded	Сходження з краю без стрибка	Falling; запускається `coyoteTime`
Falling	Jump (протягом `coyoteTime`)	Основний стрибок (Jumping)
Falling	Jump (після `coyoteTime`, заряд є)	Double Jump
Falling	Dash	Повітряний Dash, якщо заряд є
Falling	Контакт зі стіною	Wall Contact
Wall Contact	Приземлення	Grounded
Wall Contact	Ввід у бік стіни припинено	Falling (після `wallCoyoteTime`)

Wall Jump	Контакт із протилежною стіною	Wall Contact
Dash	Удар об валідну стіну	Wall Contact
Dash	Завершення тривалості в повітрі	Falling; Double Jump доступний за введенням, якщо заряд не витрачено
Dash	Завершення на землі	Grounded

Загальні правила

- Одночасно не повинні виконуватися дві несумісні механіки, які обидві керують швидкістю персонажа.
- Введення гравця під час переходів не повинно губитися без пояснюваної причини.
- Завершення Dash не повинно автоматично активувати Double Jump або Wall Jump (вони доступні лише за новим введенням гравця).
- Приземлення відновлює можливість Double Jump і повітряного Dash.
- У разі одночасного натискання кількох кнопок система повинна мати визначений порядок обробки команд.

Пропонований пріоритет команд: Dash має пріоритет над звичайним горизонтальним рухом на час виконання ривка. Wall Jump має пріоритет над звичайним Jump, коли персонаж перебуває у валідному стані контакту зі стіною.

---

6. Початкові параметри для Unity Inspector

Основні значення мають бути доступними для налаштування без необхідності щоразу змінювати код. Це дасть змогу швидко коригувати game feel під час playtest.

Категорія	Параметр	Початкове значення
Move	`maxMoveSpeed`	7
Move	`groundAcceleration`	45
Move	`groundDeceleration`	55
Move	`airAcceleration`	30
Jump	`targetJumpHeight`	2,5
Jump	`targetTimeToApex`	0,35 s
Jump	`jumpBufferTime`	0,10 s
Jump	`coyoteTime`	0,10 s
Jump	`fallGravityMultiplier`	1,5
Jump	`maxFallSpeed`	16
Jump	`jumpCutGravityMultiplier`	3
Double Jump	`doubleJumpVerticalSpeed`	11,5

Double Jump	`maxAirJumps`	1
Dash	`dashDuration`	0,16 s
Dash	`dashSpeed`	22
Dash	`maxAirDashes`	1
Dash	`dashCooldown`	0,30 s *(наземний)*
Dash	`dashGravityScale`	0
Dash	`dashExitSpeed` / `dashExitDecayTime`	8 / 0,20 s
Dash	`dashBufferTime`	0,10 s
Wall Jump	`wallJumpHorizontalSpeed`	8
Wall Jump	`wallJumpVerticalSpeed`	13
Wall Jump	`wallJumpControlLockTime`	0,08 s
Wall Contact	`wallSlideSpeed`	2

Wall Contact	`wallCoyoteTime`	0,10 s
Wall Contact	`allowSameWallRepeat`	false

Це стартові значення для тестування, а не фінально збалансовані характеристики.

---

7. Вимоги до playable prototype

7.1. Призначення

Прототип потрібен для перевірки руху та його відчуттів, а не для створення фінальної гри.

7.2. Необхідні елементи

- Один керований персонаж.
- Горизонтальний рух.
- Основний стрибок.
- Подвійний стрибок.
- Dash.
- Wall Jump.
- Фізичні колайдери персонажа та середовища.
- Камера, яка стежить за персонажем.
- Тестові платформи та стіни.
- Можливість повторити спробу після невдалого проходження.
- Кінцева точка тестової ділянки.

7.3. Респавн, чекпоінти та Falls

- **Провал:** внизу кожної ділянки є зона смерті (туманна прірва, шар `Hazard`). Контакт із нею або падіння нижче `killY` = смерть.
- **Респавн:** за 0,4–0,6 с (fade) персонаж з’являється на останньому чекпоінті; усі швидкості, таймери, та буфер вводу скидаються.
- **Falls:** лічильник падінь збільшується на 1 при кожному респавні; відображається на екрані. Таймер забігу при респавні **не** скидається.
- **Restart:** клавіша перезапускає весь тест (Falls і таймер - в нуль).
- **Чекпоінти:** по одному в кінці кожної зони (початок зони 1 - стартова точка); активуються при проходженні й зберігаються до Restart.

- **Фініш:** кінцева платформа з тригером; показує фінальний час і кількість Falls.

---

8. Структура тестового рівня

Тестовий рівень має змушувати гравця застосовувати кожну з механік у реальних ігрових ситуаціях.

Зона 1 - The Overgrown Entr

Отут добре було б схемку намалювати в маштабі 1:1 і на ній показати розміри персонажа, параметри руху відносно нього. Так утворюється верхньорівневий опис, що підійде під будь який двигун, розмір асетів і реалзацію стрибка. Заодно на схемці роставити перешкоди і зони

Призначення: перевірити Move

Елементи:

- широка стартова платформа;
- невеликі проміжки;
- платформи різної висоти;
- достатньо простору для розгону та гальмування.

Перевіряємо, чи легко контролювати швидкість і чи можна точно приземлятися.

Розміри: стартова платформа 10 units; проміжки 2, 3 і 4 units (максимум Jump ≈ 4,4); уступи заввишки 1, 1,5 і 2,2 units; ширина платформ приземлення ≥ 3 units; чекпоінт у кінці зони.

Зона 2 - The Sinking Marsh

Призначення: перевірити Double Jump.

Елементи:

- високі платформи;
- проміжки, які не можна подолати одним стрибком;
- платформи, на які потрібно приземлятися з корекцією траєкторії.

Перевіряємо, чи Double Jump дійсно відкриває нові можливості та чи не робить стрибок надто потужним.

Розміри: проміжок **6,0 units** (Jump ≈ 4,4 < 6,0 < Jump + Double Jump ≈ 7,0, запас ≈ 14%); уступ заввишки **3,6 units** (2,5 < 3,6 < 4,1, запас ≈ 12%); ширина платформ приземлення ≥ 2,5 units; внизу Hazard.

Зона 3 - The Rootbound Passage

Призначення: перевірити Wall Jump.

Елементи:

- вертикальний прохід;
- стіни з обох боків;
- проміжні платформи;
- ділянки, де необхідно змінювати горизонтальний напрямок після відштовхування.

Перевіряємо стабільність визначення стін і можливість контролювати рух після Wall Jump.

Розміри: шахта шириною **3,0 units** і висотою **9,0 units**; стіни шару `Wall` з обох боків на всю висоту (правило чергування стін 4.5 робить обидві стіни обов'язковими); дві проміжні вузькі платформи шириною 1 unit на висоті ≈ 3 і ≈ 6 (з різних боків); це ≈ 3–4 відскоки. Чекпоінт нагорі.

Зона 4 - The Broken Crossing

Призначення: перевірити Dash.

Елементи:

- широкі проміжки між платформами;
- вузькі майданчики для приземлення;
- можливість виконати Dash у різних напрямках.

Перевіряємо дистанцію ривка, відновлення керування та можливість приземлятися після Dash.

Розміри: тунель зі стелею на висоті **2,2 units** над підлогою й проміжком **5,5 units** (під стелею Jump ≈ 3,2 по горизонталі, Dash додає ≈ 3,5 → ≈ 6,7, запас ≈ 18%; Double Jump під стелею не допомагає); ширина вузького майданчика приземлення 1,5 units. *Валідація в прототипі:* якщо тунель проходиться без Dash, опустити стелю до 2,0 або збільшити проміжок до 6,0. Додатково: уступ нагорі, доступний лише діагональним Dash (опційно).

Зона 5 - The Ancient Glade

Призначення: перевірити комбінації всіх механік.

Приклад послідовності:

1. Звичайний стрибок із платформи.
2. Double Jump для набору висоти.

3. Dash для подолання широкого проміжку.
4. Wall Jump для підйому вздовж стіни.
5. Приземлення на фінальну платформу.

Послідовність повинна бути складною настільки, щоб перевірити взаємодію механік, але не настільки, щоб гравець не міг зрозуміти причину невдачі.

Розміри: проміжок **9,0 units** (потрібні всі три: Jump + Double Jump + Dash ≈ 10,5, запас ≈ 14%); приземлення на стіну шахти 3,0 × 5,0 units (Dash → Wall Contact); два Wall Jump вгору, потім фінальна платформа з Finish. Чекпоінт перед проміжком.

---

9. План playtest

№	Тест	Очікуваний результат	Ознака проблеми
1	Рух і зупинка	Персонаж швидко реагує та плавно зупиняється	Надмірне ковзання
2	Стрибок із коротким і довгим натисканням	Різниця у висоті помітна	Висота майже не змінюється
3	Корекція траєкторії в повітрі	Гравець може змінити місце приземлення	Рух у повітрі занадто слабкий
4	Double Jump	Персонаж досягає додаткової висоти	Механіка майже не відрізняється від першого стрибка
5	Dash через проміжок	Ривок дозволяє подолати відстань	Персонаж не долітає або перелетить платформу

6	Wall Jump	Персонаж піднімається вздовж стін	Застрягання або неправильний напрямок відштовхування
7	Dash із приземленням	Гравець відновлює контроль і може приземлитися	Різка втрата керування
8	Комбінація механік	Персонаж проходить складну секцію	Команди конфліктують або губляться
9	Повторне проходження	Результат руху передбачуваний	Однакові дії дають незрозуміло різний результат
10	Wall Jump двічі від тієї самої стіни	Другий відскок блокується, з протилежної стіни працює	Блокується обидві стіни або не блокується жодна
11	Падіння й респавн	Falls +1, респавн на чекпоінті зі скинутими зарядами	Заряди чи швидкість залишаються після респавну

9.1. Як документувати результати

Для кожного тесту:

- що саме перевірялося;
- що зробив гравець;
- який результат він отримав;
- що відчувалося неправильно;
- яку зміну пропонуєш;
- чи покращився результат після зміни.

---

10. Технічне завдання

Реалізувати систему руху персонажа для 2D-платформера Mossbound відповідно до концепції Fluid Platforming.

Система має підтримувати п’ять механік: Move, Jump, Double Jump, Dash і Wall Jump.

Вимоги до реалізації

- Реалізувати горизонтальний рух із налаштовуваними прискоренням, гальмуванням і максимальною швидкістю.
- Реалізувати основний стрибок із можливістю контролювати висоту через тривалість натискання.
- Реалізувати один додатковий стрибок у повітрі.
- Реалізувати Dash із налаштовуваними напрямком, швидкістю та тривалістю.
- Реалізувати Wall Jump із відштовхуванням угору та від стіни.
- Забезпечити коректні переходи між усіма механіками.
- Реалізувати відновлення Double Jump і повітряного Dash після приземлення.
- Забезпечити стабільне визначення землі та стін.
- Надати доступ до основних параметрів через Unity Inspector.
- Забезпечити можливість тестування механік у спільній демонстраційній сцені.
- Реалізувати стан Wall Contact із правилом чергування стін.
- Реалізувати респавн, чекпоінти, лічильник Falls і Restart.
- Усі швидкості й розміри - в Unity units при PPU = 16.

Вимоги до game feel

- Керування повинно швидко реагувати на введення.
- Рух на землі має бути плавним, але не надмірно інерційним.
- Стрибок повинен мати передбачувану траєкторію.
- Гравець повинен мати достатній контроль у повітрі.
- Dash повинен відчуватися як окрема коротка механіка.
- Після Dash керування має повертатися без різкого небажаного стрибка швидкості.
- Wall Jump повинен забезпечувати стабільне відштовхування та подальшу корекцію траєкторії.

Критерії готовності

- Усі п’ять механік працюють.
- Механіки можна комбінувати відповідно до таблиці переходів.

- Після приземлення відновлюються потрібні заряди.
- Немає очевидних конфліктів між рухом, Dash і стрибками.
- Параметри можна змінювати через Inspector.
- Усі механіки перевірені в тестовій сцені.
- Після playtest внесено погоджені зміни.

---

11. Acceptance Criteria

Прототип вважається готовим до демонстрації, коли:

- Персонаж може рухатися в обох напрямках.
- Прискорення та гальмування налаштовуються.
- Основний стрибок працює стабільно.
- Висота стрибка залежить від тривалості натискання.
- Double Jump доступний один раз у повітрі.
- Dash виконується в заданому напрямку.
- Після Dash відновлюється звичайний контроль.
- Wall Jump працює з обох боків стіни.
- Комбінації механік працюють без очевидних конфліктів.
- Тестовий рівень вимагає застосування всіх механік.
- Є можливість повторно пройти тест після невдачі.
- Проведено playtest і зафіксовано щонайменше одну обґрунтовану ітерацію дизайну.
- Falls, респавн на чекпоінті та Restart працюють коректно.